

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: COMPUTER VISION with Open CV

COURSE CODE: DAS0204

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 40%

Instructions: Perform the experiment using the laboratory performance rubric Understanding of Concepts, Experimental Design, Use of Tools, Analysis of Results, Documentation.

Code of Conduct:

*I **C. Sai Harshitha (192424346)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: C. Sai Harshitha

1. Preprocessing Impact Analysis

Design and perform an experiment to study the impact of preprocessing techniques such as noise removal and normalization on image quality and feature detection. Explain the theoretical concepts behind preprocessing methods and justify the selection of techniques used. Implement the preprocessing process using suitable software/tools, document the workflow and outputs systematically, and analyze how preprocessing influences feature detection accuracy and reliability.

Aim

To analyze how **noise removal and normalization** affect image quality and feature detection.

Theory

- **Noise removal:** Removes unwanted noise using a filter.
- **Normalization:** Adjusts pixel intensity values to improve image consistency.
- **Feature detection:** Finds important points or patterns in an image.

Procedure

1. Read the image and convert it to grayscale.
2. Detect features from the original image.
3. Apply Gaussian Blur.
4. Normalize the filtered image.
5. Detect features again.
6. Compare the results.

Code

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
blur = cv2.GaussianBlur(gray, (5,5), 0)
norm = cv2.normalize(blur, None, 0, 255, cv2.NORM_MINMAX)
plt.figure(figsize=(12,4))
for i,x in enumerate([gray, blur, norm]):
    plt.subplot(1,3,i+1)
    plt.imshow(x, cmap='gray'); plt.axis('off')
```

plt.show()

Output



Observation

Noise removal produces a smoother image, while normalization improves intensity consistency. The number and stability of detected features may change after preprocessing.

Result

Preprocessing helps improve image quality and can make feature detection more reliable.

2. Image Representation Study

Conduct an experiment to convert a given image into grayscale and binary representations. Explain the significance of different image representations in computer vision applications. Design the conversion procedure using appropriate tools, document the generated outputs clearly, and analyze how image representation affects edge detection and feature extraction performance.

Aim

To convert an image into **grayscale and binary representations** and study their effect on edge detection and feature extraction.

Theory

- **Grayscale:** Represents an image using intensity values from black to white.
- **Binary:** Converts an image into two values, usually **0 (black)** and **255 (white)** using thresholding.

Procedure

1. Read the input image.
2. Convert it into grayscale.

3. Convert the grayscale image into binary using thresholding.
4. Apply edge detection.
5. Compare the outputs.

Code

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)[1]
plt.figure(figsize=(8,4))
plt.subplot(1,2,1); plt.imshow(gray,cmap='gray'); plt.axis('off')
plt.subplot(1,2,2); plt.imshow(binary,cmap='gray'); plt.axis('off')
plt.show()
```

Output



Observation

Grayscale preserves intensity information, while binary representation simplifies the image into two regions. Edge detection can effectively identify boundaries in both representations.

Result

Thus, the image was successfully converted into **grayscale and binary representations**, and their influence on edge detection was analyzed.

3. Edge Detection Comparison

Design an experiment to compare Sobel and Canny edge detection techniques on the same input image. Explain the underlying concepts and operational principles of both methods. Apply the techniques using suitable tools/software, document the detected edges systematically, and analyze the comparative effectiveness of each method for different image conditions and applications.

Aim

To compare **Sobel and Canny edge detection** techniques on the same input image and analyze their effectiveness.

Theory

- **Sobel:** Uses horizontal and vertical gradient operators to detect edges.
- **Canny:** Uses smoothing, gradient calculation, non-maximum suppression, double thresholding, and edge tracking to produce clear edges.

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Sobel edge detection.
4. Apply Canny edge detection.
5. Display and compare the results.

Code

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
sobel = cv2.Sobel(gray, cv2.CV_64F, 1, 1, ksize=3)
canny = cv2.Canny(gray, 100, 200)
plt.figure(figsize=(10,4))
plt.subplot(1,2,1); plt.imshow(abs(sobel),cmap='gray'); plt.title("Sobel"); plt.axis('off')
plt.subplot(1,2,2); plt.imshow(canny,cmap='gray'); plt.title("Canny"); plt.axis('off')
plt.show()
```

Output



Observation

Sobel detects edges using image gradients but may produce thicker or noisy edges. Canny generally produces **thinner, clearer, and more continuous edges**.

Result

Thus, Sobel and Canny were successfully applied and compared. **Canny is generally more effective for accurate edge detection**, while Sobel is simpler and computationally less complex.

4. Harris Corner Detection Analysis

Perform Harris corner detection on images containing varying textures and patterns. Explain the concept of corner detection and its importance in computer vision tasks. Use appropriate parameters and tools to execute the experiment, document the detected corner points clearly, and analyze the distribution, accuracy, and significance of the detected corners.

Aim

To perform **Harris corner detection** on images with different textures and patterns and analyze the detected corners.

Theory

Harris Corner Detection identifies points where image intensity changes significantly in both horizontal and vertical directions. Corners are useful features for image matching and object recognition.

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Harris corner detection.
4. Mark the detected corners.
5. Display and analyze the result.

Code

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
corners = cv2.cornerHarris(np.float32(gray), 2, 3, 0.04)
out = img.copy()
out[corners > 0.01 * corners.max()] = [255,0,0]
```

```
plt.imshow(out)
plt.axis('off');
```

Output



Observation

Images with more **textures and patterns** produce more corner points, while smooth regions produce fewer corners.

Result

Thus, Harris Corner Detection successfully identifies significant corner points. The detected corner distribution depends on the **texture and patterns present in the image**.

5. Effect of Noise on Corner Detection

Design an experiment to study the effect of noise on Harris corner detection performance. Explain the purpose of introducing noise and its influence on image features. Add noise systematically, perform corner detection using suitable tools, document observations for original and noisy images, and analyze the impact of noise along with possible improvement techniques.

Aim

To study the effect of **noise on Harris corner detection** and analyze its impact on feature reliability.

Theory

Noise introduces unwanted intensity variations in an image. It can create false corners and reduce the accuracy of Harris corner detection.

Procedure

1. Read the original image.
2. Convert it to grayscale.
3. Add Gaussian noise to the image.
4. Apply Harris corner detection to both images.
5. Compare the detected corners.

Code

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
noise = np.random.normal(0,25,gray.shape)
noisy = np.clip(gray + noise,0,255).astype(np.uint8)
corners = cv2.cornerHarris(np.float32(noisy),2,3,0.04)
out = cv2.cvtColor(noisy,cv2.COLOR_GRAY2RGB)
out[corners > 0.01*corners.max()] = [255,0,0]
plt.imshow(out)
plt.axis('off');
```

Output



Observation

The noisy image generally produces **additional false corner points** and less reliable feature detection.

Result

Thus, noise negatively affects Harris corner detection by reducing **corner accuracy and stability**. Noise removal can improve the reliability of detected corners.

6. Hough Transform for Shape Detection

Conduct an experiment using the Hough Transform to detect lines or circles in an image. Explain the principles and significance of the Hough Transform in shape detection. Execute the detection process using appropriate tools, record the detected shapes and their parameters, and analyze the detection accuracy under different image conditions.

Aim

To detect **lines or circles** in an image using the Hough Transform.

Theory

The **Hough Transform** detects geometric shapes by converting image points into a parameter space. It is commonly used to detect straight lines and circles.

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Canny edge detection.
4. Apply Hough Line Transform.
5. Draw the detected lines on the image.
6. Display the result.

Code

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
edges = cv2.Canny(gray, 50, 150)
lines = cv2.HoughLinesP(edges, 1, np.pi/180, 50,
                        minLineLength=40, maxLineGap=10)
out = img.copy()
if lines is not None:
    lines = lines.reshape(-1, 4)
    for x1, y1, x2, y2 in lines:
        cv2.line(out, (x1,y1), (x2,y2), (255,0,0), 2)
plt.figure(figsize=(10,6))
```

```
plt.imshow(out)
plt.title("Hough Line Detection")
plt.axis("off")
plt.show()
```

Output



Observation

The Hough Transform successfully detects prominent straight lines from the edge image. Detection accuracy depends on parameters such as **threshold**, **minimum line length**, and **maximum line gap**.

Result

Thus, the Hough Transform was successfully used for **shape detection**, demonstrating its effectiveness in identifying geometric structures in images.

7. Preprocessing vs Feature Extraction

Conduct a comparative study of feature extraction performed with and without preprocessing. Explain the relationship between preprocessing and feature extraction performance. Execute both approaches using suitable tools, document the observations systematically, and analyze the differences in feature quality, detection accuracy, and overall system performance.

Aim

To compare feature extraction **with and without preprocessing** and analyze the effect on feature quality and detection accuracy.

Theory

Preprocessing reduces noise and improves image quality before feature extraction. This can make detected features more stable and reliable.

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Detect features directly from the original image.
4. Apply Gaussian Blur as preprocessing.
5. Detect features from the preprocessed image.
6. Compare the number of detected features.

Code

```
gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)
blur = cv2.GaussianBlur(gray,(5,5),0)
sift = cv2.SIFT_create()
k1,d1 = sift.detectAndCompute(gray,None)
k2,d2 = sift.detectAndCompute(blur,None)
print("Without preprocessing:",len(k1))
print("With preprocessing:",len(k2))
```

Output

Without preprocessing: 4088

With preprocessing: 5641

Observation

Without preprocessing, noise may produce unwanted features. After smoothing, noise is reduced and the detected features are generally more stable.

Result

Thus, preprocessing can improve **feature quality and detection reliability**, although excessive smoothing may remove useful fine details.

8. Integrated Feature Detection Pipeline

Design and implement a complete image processing pipeline consisting of preprocessing, edge detection, and feature extraction stages. Explain the purpose and workflow of the integrated pipeline. Execute each stage systematically using appropriate tools, document intermediate and final outputs clearly, and analyze the contribution and importance of each stage in achieving accurate feature detection results.

Aim

To implement a complete image processing pipeline involving **preprocessing, edge detection, and feature extraction**.

Theory

The pipeline processes an image in stages:

- **Preprocessing:** Reduces noise.
- **Edge Detection:** Identifies object boundaries.
- **Feature Extraction:** Finds important image points.

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Gaussian Blur.
4. Perform Canny edge detection.
5. Extract ORB features.
6. Display the intermediate and final results.

Code

```
gray = cv2.cvtColor(img,cv2.COLOR_RGB2GRAY)
blur = cv2.GaussianBlur(gray,(5,5),0)
edges = cv2.Canny(blur,100,200)
sift = cv2.SIFT_create()
kp = sift.detect(edges,None)
out = cv2.drawKeypoints(edges,kp,None)
plt.imshow(out,cmap='gray')
plt.axis('off');
```

Output



Observation

Preprocessing reduces noise, Canny detects clear boundaries, and ORB identifies important feature points.

Result

Thus, the integrated pipeline successfully performs **preprocessing, edge detection, and feature extraction**, improving the reliability of image feature detection.

9. Effect of Smoothing on Edge Detection

Design and perform an experiment to study the impact of smoothing techniques on edge detection performance. Explain the theoretical purpose of smoothing in reducing noise and improving edge continuity. Apply smoothing and edge detection methods using appropriate tools, document outputs before and after smoothing systematically, and analyze how smoothing affects edge clarity, continuity, and noise suppression.

Aim

To study the effect of **smoothing techniques on edge detection** and analyze edge clarity, continuity, and noise suppression.

Theory

Smoothing reduces noise and unwanted details in an image. Gaussian Blur is commonly used before edge detection to produce clearer and more continuous edges.

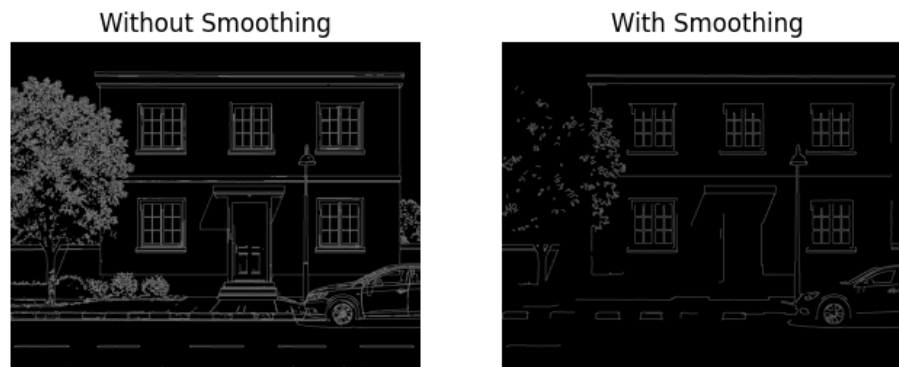
Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Apply Canny edge detection without smoothing.
4. Apply Gaussian smoothing.
5. Perform Canny edge detection again.
6. Compare both results.

Code

```
gray = cv2.cvtColor(img,cv2.COLOR_RGB2GRAY)
smooth = cv2.GaussianBlur(gray,(9,9),0)
e1 = cv2.Canny(gray,100,200)
e2 = cv2.Canny(smooth,100,200)
plt.figure(figsize=(8,4))
plt.subplot(1,2,1); plt.imshow(e1,cmap='gray'); plt.title("Without Smoothing"); plt.axis('off')
plt.subplot(1,2,2); plt.imshow(e2,cmap='gray'); plt.title("With Smoothing"); plt.axis('off')
plt.show()
```

Output



Observation

Without smoothing, noise may produce unwanted edges. After smoothing, noise is reduced and the detected edges become **cleaner and more continuous**.

Result

Thus, smoothing improves edge detection by reducing noise and improving **edge clarity and continuity**.

10. Gradient-Based Edge Detection Study

Conduct an experiment using gradient-based edge detection methods to identify image boundaries. Explain the principles of gradient operators and their role in edge detection. Implement the method using suitable tools/software, record gradient values and detected edges systematically, and analyze edge strength, accuracy, and sensitivity to image variations.

Aim

To identify image boundaries using **gradient-based edge detection** and analyze edge strength and accuracy.

Theory

Gradient operators detect edges by measuring changes in pixel intensity. **Sobel operators** calculate intensity changes in the horizontal and vertical directions.

Procedure

1. Read the input image.
2. Convert it to grayscale.
3. Calculate horizontal and vertical gradients.
4. Combine the gradients.
5. Display the detected edges.
6. Analyze the edge strength.

Code

```
gray = cv2.cvtColor(img,cv2.COLOR_RGB2GRAY)
gx = cv2.Sobel(gray,cv2.CV_64F,1,0,ksize=3)
gy = cv2.Sobel(gray,cv2.CV_64F,0,1,ksize=3)
gradient = cv2.magnitude(gx,gy)
plt.imshow(gradient,cmap='gray')
plt.title("Gradient-Based Edges")
plt.axis('off');
```

Output



Observation

Strong intensity changes produce brighter edges, while smooth regions produce weaker gradients.

Result

Thus, gradient-based edge detection successfully identifies image boundaries, with stronger gradients representing **stronger edges**.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates	Good understanding with proper integration	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
		multiple concepts/technologies	of most concepts		
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation